

第三阶段个人工作小结

1、本阶段工作内容

本阶段主要完成了系统的本地集成联调，以及大模型输出数据的清洗工作。在本地运行环境方面，编写了项目的一键启动脚本（start.sh）和对应的配置（docker-compose.yml），将 Java 网关、Python 算法服务和 PostgreSQL 数据库统一封装在本地 Docker 环境中运行，方便进行全链路测试。在算法数据处理方面，修改了 chat_routes.py 中的流式输出逻辑，增加了针对 DSL 动画指令的数据清洗功能。通过引入正则表达式，对大模型生成的内容进行结构化提取和格式校验，确保前端接收到的 JSON 数据格式符合协议要求。

2、遇到的问题以及解决思路

在本地进行全系统联调时，同时启动 Flutter 前端模拟器、Java 后端、Python 算法端和数据库容器，导致电脑内存占用过高。Python 端在加载模型处理词向量时会出现内存峰值，导致 Docker 引擎触发 OOM 机制，关闭了 Python 和数据库进程。为了解决这个问题，我们对各容器的资源占用进行限制。首先在 Java 服务的 Dockerfile 中增加了 -Xms512m -Xmx512m 的 JVM 启动参数以限制内存上限。其次，修改了 Python 端的词向量并发处理代码，去除了多进程池，改用单线程异步处理，降低了运行时的峰值内存消耗，保证了本地联调的稳定性。

此外，在联调测试中我们还发现，大模型在生成复杂的 DSL 渲染指令时，偶尔会输出非标准格式（例如外层包裹了 Markdown 代码块标记，或因长度限制导致 JSON 右括号未闭合）。由于前端是强类型解析，格式错误会导致页面渲染失败。我在 Python 端增加了一层数据清洗拦截器。每次获取模型输出后，先用正则提取出 {} 或 [] 内的纯 JSON 内容。如果遇到截断的数据，利用 Python 的异常捕获机制尝试自动补全 } 和]。对于无法修复的错误数据，将其拦截丢弃，并向前端返回一段默认的纯文本指令，避免前端界面崩溃。

3、下一阶段计划

目前核心代码开发已经结束。下一阶段的主要任务是和全组同学一起构思系

统的演示操作流程, 统稿并完善项目的补充文档(如环境配置说明、测试报告等)。
同时, 共同制作期末答辩所需的 PPT, 并配合完成系统最终操作演示视频的录制。